University of Niagara Falls

Data Analytics MSc

Assignment 2

Supply Chain Intelligent Database System

SQL Databases Final Project (CPSC-500-16)

Instructor: Prof. William Pourmajidi

Groups Members:

- **Nicolas Ricardo Ysa Pelaez**
- **Sravani Maguluru**
- **Diego Andres Tenecela Alcivar**
- **Jose Alberto Martinez Morales**

Date Submitted: June 16, 2026

Table of Contents

Chapter 1 System Overview

1.1 Project Domain

- Brief description of the supply chain domain.
- Why this domain requires intelligent database systems
- Real-world relevance (inventory, suppliers, shipments, orders)

This project is developed within the **supply chain and logistics domain**, a complex operational environment responsible for coordinating the flow of goods from suppliers to warehouses and ultimately to customers. Modern supply chains involve multiple interconnected processes such as procurement, inventory management, warehousing, transportation, and order fulfillment. Each of these processes generates large volumes of structured and semi-structured data that must be stored, validated, and analyzed efficiently.

The domain requires **intelligent database systems** because supply chains operate in real time and depend on accurate, integrated information to make decisions. Organizations must track supplier performance, monitor inventory levels, manage shipments, optimize delivery routes, and respond quickly to disruptions. Traditional static databases are not sufficient; instead, systems must support dynamic data, flexible schemas, and analytics-ready structures.

This project models the operations of **Niagara Fulfillment Network (NFN)**, a fictional company inspired by Amazon's *Fulfillment by Amazon (FBA)* model. In this business model, suppliers sell products **to NFN**, not directly to customers. NFN receives goods through **purchase orders** and **shipments**, stores them in its own **warehouses**, and manages all downstream logistics. Customers purchase products **from NFN**, and the company handles order processing, picking, packing, and delivery through its internal routing system.

This mirrors real-world supply chain operations and is reflected in the database tables:

- **Suppliers → Purchase_Orders → Shipments**
- **Warehouses → Inventory**
- **Customers → Orders → Order_Items**
- **Delivery_Routes → final delivery**

Together, these components create a realistic, analytics-ready supply chain environment.

1.2 System Purpose

The purpose of this intelligent database system is to support the **end-to-end operations of the Niagara Fulfillment Network (NFN)**, from procurement to final delivery. The database is designed to store, manage, and analyze data across multiple operational areas, enabling efficient decision-making and performance monitoring.

The system supports several key operations:

Procurement

- Managing suppliers (Suppliers)
- Creating and tracking purchase orders (Purchase_Orders)
- Recording ordered products and costs (Purchase_Order_Items)

Warehousing

- Managing warehouse locations and capacity (Warehouses)
- Tracking inventory levels by product and warehouse (Inventory)
- Storing warehouse configuration metadata (XML)

Order Fulfillment

- Capturing customer information (Customers)
- Recording customer orders (Orders)
- Linking orders to products (Order_Items)
- Tracking order status from processing to delivery

Logistics and Routing

- Managing shipments from suppliers to warehouses (Shipments)
- Recording received quantities (Shipment_Items)
- Storing delivery route structures (Delivery_Routes)

This system provides a unified platform for operational data, enabling NFN to streamline workflows, reduce errors, and support advanced analytics.

1.3 High-Level Architecture

The architecture integrates **structured relational data** with **semi-structured JSON and XML fields**, reflecting the flexibility required in modern supply chain environments.

Structured Data (Relational Tables)

The core of the system is built on **12 normalized tables**, including:

- Suppliers
- Warehouses
- Products
- Customers
- Purchase_Orders
- Purchase_Order_Items
- Inventory
- Orders
- Order_Items
- Shipments
- Shipment_Items
- Delivery_Routes

These tables enforce referential integrity through primary and foreign keys, ensuring consistent and reliable data relationships.

Semi-Structured Data (JSON and XML Fields)

To support flexibility and real-world variability, the system incorporates:

JSON Fields

Used in:

- Supplier metadata
- Product attributes
- Customer profiles
- Purchase order details
- Delivery route structures

XML Fields

Used in:

- Warehouse configuration
- Order metadata

Support for Analytics and Machine Learning

The architecture is intentionally designed to support downstream analytics and ML applications. The system enables:

- Demand forecasting
- Supplier reliability classification
- Customer segmentation
- Route optimization
- Inventory prediction

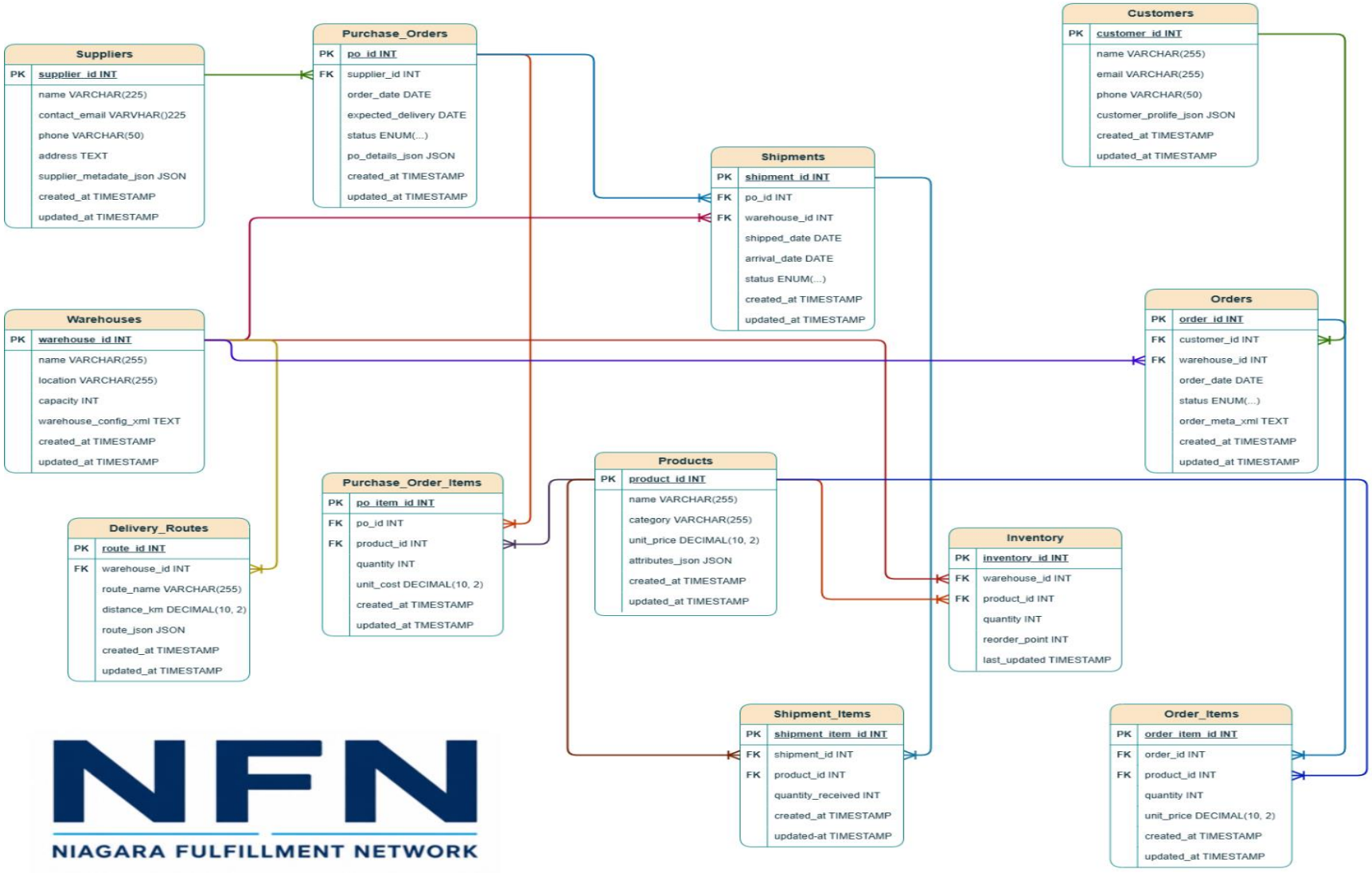
By combining relational integrity with flexible semi-structured fields, the system becomes a powerful foundation for intelligent decision-making.

Chapter 2 Database Design

2.1 ER Diagram

The Entity–Relationship (ER) diagram for the Niagara Fulfillment Network (NFN) database was created using **Draw.io (diagrams.net)**. This tool allowed us to manually design and refine the relational structure based on the finalized SQL schema. To ensure clarity and readability, the ER diagram is presented on a **dedicated full page**, allowing the tables, attributes, and relationships to appear larger and easier to interpret. This layout helps reviewers clearly see primary keys, foreign keys, and cardinalities without visual clutter. The exported image/PDF of the diagram is included on the following page.

Niagara Fulfillment Network
Entity-Relationship Diagram



NFN
NIAGARA FULFILLMENT NETWORK

2.2 Schema Overview

Below is an overview of all tables in the supply_chain database and their purposes:

- **Suppliers:** Stores supplier master data, contact information, address, and flexible metadata about each supplier.
- **Warehouses:** Represents physical warehouse locations, their capacity, and configuration details.
- **Products:** Contains product catalog information, including category, pricing, and dynamic attributes.
- **Customers:** Stores customer identities, contact details, and behavioral/profile information.
- **Purchase_Orders:** Records purchase orders placed by NFN to suppliers, including dates, status, and additional details.
- **Purchase_Order_Items:** Line-item details for each purchase order, linking products to purchase orders with quantities and unit costs.
- **Inventory:** Tracks on-hand stock levels of each product in each warehouse, including reorder thresholds.
- **Orders:** Represents customer orders placed with NFN, including order date, status, and associated warehouse.
- **Order_Items:** Line-item details for each customer order, linking products to orders with quantities and prices.
- **Shipments:** Represents shipments from suppliers to warehouses, including shipped and arrival dates and shipment status.
- **Shipment_Items:** Line-item details for each shipment, recording which products and quantities were received.
- **Delivery_Routes:** Stores delivery routes associated with warehouses, including distance and route structure.

2.3 Normalization Rationale

The schema is designed to follow **Third Normal Form (3NF)**:

- **No repeating groups:** Each table represents a single, well-defined entity (e.g., Suppliers, Products, Orders).
- **No partial dependencies:** All non-key attributes depend on the full primary key (e.g., in Purchase_Order_Items, quantity and unit_cost depend on the full key po_item_id / (po_id, product_id conceptually).
- **No transitive dependencies:** Attributes such as supplier contact details are stored only in Suppliers, not duplicated in Purchase_Orders or Shipments.

Redundancy elimination:

- Supplier details are stored only in Suppliers and referenced via supplier_id.
- Product details are stored only in Products and referenced in Purchase_Order_Items, Order_Items, and Shipment_Items.
- Customer details are stored only in Customers and referenced in Orders.
- Warehouse details are stored only in Warehouses and referenced in Inventory, Orders, Shipments, and Delivery_Routes.

Bridge tables for many-to-many relationships:

Many real-world relationships are naturally many-to-many. To keep the schema normalized, these are resolved using bridge (junction) tables:

- Purchase_Order_Items connects **Purchase_Orders** and **Products**.
- Shipment_Items connects **Shipments** and **Products**.
- Order_Items connects **Orders** and **Products**.

This design avoids duplication of product data inside orders or shipments and allows flexible association of multiple products with multiple orders/shipments.

2.4 Relationship Justification

Explain why each relationship type was chosen:

- One-to-Many
    - Suppliers -> Purchase Orders
    - Warehouses -> Inventory
    - Customers -> Orders
- Many-to-Many
    - Purchase Orders <-> Products (via Purchase_Order_Items)
    - Shipments <-> Products (vis Shipment_Items)
    - Orders <-> Products (via Order_Items)

One-to-many relationships

- **Suppliers → Purchase_Orders** One supplier can be associated with many purchase orders over time, but each purchase order belongs to exactly one supplier. This reflects the business reality that NFN issues purchase orders to specific suppliers.
- **Warehouses → Inventory** One warehouse can store many different products, but each inventory record refers to a single warehouse–product combination. This supports per-warehouse stock tracking.
- **Customers → Orders** One customer can place many orders, but each order is associated with exactly one customer. This allows NFN to analyze customer behavior and order history.

Many-to-many relationships (via bridge tables)

- **Purchase_Orders ↔ Products (via** Purchase_Order_Items**)** A single purchase order can contain multiple products, and the same product can appear in many different purchase orders. Purchase_Order_Items captures quantities and unit costs per product per purchase order.
- **Shipments ↔ Products (via** Shipment_Items**)** A shipment can include multiple products, and a product can be shipped in multiple shipments. Shipment_Items records the quantity of each product received in each shipment.

- **Orders ↔ Products (via** Order_Items**)** A customer order can contain multiple products, and a product can appear in many different orders. Order_Items stores the quantity and unit price per product per order.

These bridge tables provide flexibility, maintain normalization, and support detailed analytics online-item level data.

2.5 Use of JSON Fields

JSON is used to model **flexible, semi-structured attributes** that vary significantly between records and may evolve over time. This avoids constant schema changes and supports richer analytics.

- **Suppliers (**supplier_metadata_json**):** Stores metadata such as certifications, risk scores, delivery preferences, or performance indicators.

  Benefits:

  - **Flexibility:** Different suppliers can have different metadata structures.
  - **Avoiding schema explosion:** No need for many nullable columns for rarely used attributes.
  - **ML support:** Supplier risk, performance tags, and ratings can be extracted as features.

- **Products (**attributes_json**):** Stores product-specific attributes such as dimensions, weight, color, material, or packaging type.

  Benefits:

  - **Flexibility:** Different product categories require different attributes.
  - **Avoiding schema explosion:** Prevents dozens of attribute columns.
  - **ML support:** Product attributes can be used for demand forecasting, recommendation, or clustering.

- **Customers (**customer_profile_json**):** Stores preferences, loyalty tier, segmentation tags, and behavioral indicators.

Benefits:

- **Flexibility:** Profiles can evolve as new behavioral signals are added.
- **ML support:** Enables customer segmentation, churn prediction, and personalization.

- **Purchase_Orders (**po_details_json**):** Stores special instructions, payment terms, or additional metadata related to each purchase order.

  Benefits:

  - **Flexibility:** Different orders may require different instructions.
  - **ML support:** Can be used to analyze patterns in special handling or terms.

- **Delivery_Routes (**route_json**):** Stores route structures such as GPS coordinates, waypoints, or step-by-step instructions.

  Benefits:

  - **Flexibility:** Routes can vary in complexity and structure.
  - **ML support:** Useful for route optimization, travel time estimation, and logistics analytics.

Overall, JSON fields allow the system to remain normalized while still capturing rich, evolving business data that is highly valuable for analytics and machine learning.

2.6 Use of XML Fields

XML is used for **hierarchical configuration and human-readable metadata**, especially where structured documents or legacy integration are relevant.

- **Warehouses (**warehouse_config_xml**):** Stores configuration details such as storage zones, temperature-controlled areas, safety rules, or layout information.

  Reasons:

  - **Hierarchical configuration:** Warehouse layouts and zones are naturally hierarchical.
  - **Human-readable metadata:** XML can be easily inspected and edited as a document.

    - **Legacy compatibility:** Many enterprise systems still exchange configuration data in XML.

- **Orders (**order_meta_xml**):** Stores packaging instructions, fragile flags, delivery notes, or special handling requirements.

  Reasons:

    - **Hierarchical configuration:** Instructions may contain nested elements (e.g., per-item or per-package notes).
    - **Human-readable metadata:** XML is suitable for generating or consuming document-like order instructions.
    - **Legacy compatibility:** XML is common in EDI and B2B integrations.

Using XML here allows the system to handle structured, document-style metadata without complicating the relational schema.

2.7 Key Design Decisions

- **CHECK constraints:** Used to enforce basic business rules, such as non-negative quantities (quantity >= 0), positive capacity, and valid distances. This prevents invalid data from entering the system and ensures data quality at the database level.
- **ENUM workflow states:** Columns like status in Purchase_Orders, Orders, and Shipments use ENUM to restrict values to valid workflow states (e.g., Pending, Shipped, Delivered, Cancelled). This enforces consistent process states and simplifies reporting and analytics on order and shipment lifecycles.
- **ON DELETE CASCADE:** Foreign keys use ON DELETE CASCADE in tables such as Purchase_Order_Items, Shipment_Items, Order_Items, and Inventory. This ensures that when a parent record (e.g., a purchase order or order) is deleted, all dependent line items are automatically removed, preventing orphaned records and preserving referential integrity.
- **UNIQUE constraints:** In Inventory, the combination (warehouse_id, product_id) is enforced as unique. This guarantees that there is at most one inventory record per product per warehouse, simplifying queries and ensuring consistent stock representation.
- **Composite key logic (Inventory):** Although Inventory uses a surrogate primary key (inventory_id), the **logical composite key** is (warehouse_id, product_id), enforced via a

UNIQUE constraint. This design combines the simplicity of an auto-increment primary key with the business rule that each warehouse–product pair should appear only once.

These design decisions collectively ensure that the database is **robust, consistent, and analytics-ready**, while accurately modeling the Niagara Fulfillment Network's supply chain operations.

Chapter 3 Data Definition Language (DDL)

3.1 Table Creation Scripts

This section describes the structure and purpose of all tables used in the Niagara Fulfillment Network (NFN) Supply Chain Management System. All CREATE TABLE statements, including constraints and data types, are provided in the attached file **NFN_SupplyChain_DDL.sql**. (see Screenshot A1)

1. Suppliers

Stores supplier master data, including flexible metadata in JSON format (e.g., rating, reliability, region). Includes timestamps for auditing and automatic updates. Constraints ensure unique supplier identity and prevent duplicate contact emails.

2. Warehouses

Stores warehouse names, locations, capacity limits, and XML-based configuration data. Capacity constraints ensure data integrity. Timestamps track updates to warehouse configuration.

3. Products

Represents the NFN product catalog. Includes product attributes stored in JSON format for flexibility (e.g., size, color, brand). Constraints ensure valid pricing and prevent negative values.

4. Customers

Stores customer master data, including behavioral profiles in JSON format. Unique email constraints prevent duplicate customer accounts. Timestamps support customer lifecycle tracking.

5. Purchase Orders

Represents procurement orders placed to suppliers. Includes expected delivery dates, status tracking, and JSON-based PO details. Constraints ensure valid supplier references and enforce logical delivery dates.

6. Inventory

Tracks stock levels per warehouse and product. Includes reorder points and automatic timestamp updates. A unique constraint ensures each warehouse–product combination appears only once.

7. Orders

Stores customer orders, including XML metadata such as priority, channel, and fulfillment notes. Status values were expanded to support NFN's operational workflow (e.g., Processing, Completed, Cancelled). Foreign keys ensure valid customer and warehouse references.

8. Purchase Order Items

Stores line-item details for purchase orders. Includes unit cost, quantity, and timestamps. Constraints ensure valid product references and prevent negative quantities.

9. Shipments

Tracks inbound shipments from suppliers to NFN warehouses. Includes shipped and arrival dates, shipment status, and validation rules to ensure logical date sequences. Supports delivery-delay analytics.

10. Shipment Items

Stores item-level details for shipments, including quantities received. Constraints prevent negative quantities and ensure valid shipment and product references.

11. Order Items

Stores line-item details for customer orders. Includes unit price, quantity, and timestamps. Constraints ensure valid product references and prevent invalid pricing or quantities.

12. Delivery Routes

Stores delivery route information, including distance and route metadata in JSON format. Supports last-mile delivery optimization and routing analytics.

3.2 Views

Two analytical views were created to support reporting and machine learning workflows. The full SQL definitions are included in NFN_SupplyChain_DDL.sql.

Order Summary View

Purpose: The Order Summary View provides a consolidated, order-level snapshot of customer orders. It aggregates key metrics such as the total number of items in each order and the total monetary value of the order. This view is frequently used in dashboards, operational reporting, and machine learning preprocessing.

Analytical Value:

- Simplifies order-level reporting by combining data from multiple tables
- Supports revenue analysis, order size distribution, and customer behavior modeling
- Provides clean, ready-to-use features for ML tasks such as order cancellation prediction or demand forecasting

Supplier Performance View

Purpose: The Supplier Performance View summarizes key performance indicators (KPIs) for each supplier. It aggregates data such as average supplier rating, number of purchase orders, and total units supplied. This view is essential for supplier segmentation, performance monitoring, and clustering analysis.

Analytical Value:

- Enables NFN to evaluate supplier reliability and contribution
- Supports ML clustering models that group suppliers by performance
- Helps identify high-performing vs. underperforming suppliers
- Useful for procurement optimization and strategic sourcing decisions

3.3 Stored Procedures

The Niagara Fulfillment Network (NFN) system includes stored procedures to automate operational workflows and ensure consistent business logic across the database. These procedures support shipment processing, inventory management, and order fulfillment.

Stored Procedure 1. Mark Shipment as Arrived

Purpose: This procedure updates the status of a shipment to *Arrived* when it reaches the NFN warehouse. It ensures that shipment status changes are handled consistently and provides a controlled mechanism for updating inbound logistics data.

Business Value:

- Standardizes how shipment arrivals are recorded
- Supports downstream processes such as inventory updates
- Ensures data accuracy for delivery-delay analytics

Stored Procedure 2. Update Inventory After Customer Order

Purpose: This procedure deducts ordered quantities from the appropriate warehouse's inventory whenever a customer order is processed. It ensures that stock levels remain accurate and synchronized with order activity.

Business Value:

- Prevents overselling
- Maintains real-time inventory accuracy
- Supports fulfillment planning and replenishment forecasting

3.4 Triggers

Triggers were implemented to enforce business rules automatically and maintain data integrity without requiring manual intervention.

Trigger 1. Auto-Update Inventory When Shipment Arrives

Purpose: When a shipment's status changes to *Arrived*, this trigger automatically increases inventory levels based on the quantities received in the shipment.

Business Value:

- Ensures inventory is updated immediately upon arrival
- Eliminates manual data entry

- Supports accurate stock availability for order fulfillment

Trigger 2. Prevent Negative Inventory

Purpose: This trigger prevents any update that would cause inventory levels to fall below zero. If such an update is attempted, the trigger raises an error and blocks the transaction.

Business Value:

- Enforces a critical business rule
- Prevents data corruption
- Protects downstream processes such as forecasting and replenishment

3.5 Indexing Strategy

Indexes were created to improve query performance of NFN's operational and analytical workloads. Each index was chosen based on observed query patterns and the needs of machine learning datasets.

Index 1. Inventory Product Index

Purpose: Improves performance for queries joining Inventory with Products or Shipment Items.

Benefit:

- Faster stock lookups
- Improved performance for replenishment and forecasting queries

Index 2. Orders by Customer

Purpose: Optimizes queries that filter or group orders by customer.

Benefit:

- Faster customer order history retrieval
- Supports customer analytics and segmentation

Index 3. Shipments by Purchase Order

Purpose: Speeds up joins between Shipments and Purchase Orders.

Benefit:

- Faster inbound logistics reporting
- Improved performance for delivery-delay analytics

Index 4. Full-Text Index on Product Names

Purpose: Enables fast keyword search on product names.

Benefit:

- Supports search functionality in dashboards and internal tools
- Improves user experience for product lookup

Index 5. Supplier Rating Index (Generated Column)

Purpose: Since MySQL cannot index JSON values directly, a stored generated column was created to extract supplier rating from JSON. An index was then applied to this generated column.

Benefit:

- Enables fast filtering and sorting by supplier rating
- Supports supplier segmentation and clustering ML models

Chapter 4 Data Manipulation Language (DML)

The Data Manipulation Language (DML) operations for the Niagara Fulfillment Network (NFN) database include data insertion, updates, deletions, and validation queries. These operations populate the system with synthetic data, demonstrate business logic updates, and verify referential integrity across the schema.

Note: AI assistance (Microsoft Copilot) was used to generate synthetic data, JSON/XML structures, and example DML operations to ensure realism, consistency, and alignment with the NFN business model.

4.1 Data Insertion

NFN requires a realistic dataset to support analytics, machine learning, and operational testing. More than **200 synthetic records** were inserted across the 12 tables, including suppliers, warehouses, products, customers, purchase orders, shipments, orders, and inventory. (See screenshot A2)

Key characteristics of the inserted data:

- **AI-generated synthetic data** (via Microsoft Copilot) was used to simulate realistic business operations.
- **JSON fields** were populated with attributes such as:
    - Supplier ratings
    - Customer types
    - Product attributes
    - Purchase order metadata
- **XML fields** were populated with:
    - Order priority
    - Order channel

    - Warehouse configuration settings

- **Timestamps** were automatically generated to simulate real operational timelines.

These insertions ensure that NFN has a rich dataset suitable for dashboards, analytics, and machine learning model development.

Full insertion scripts are included in **NFN_SupplyChain_DML.sql**.

4.2 Update Operations

This section demonstrates how NFN updates operational data, including traditional updates, JSON modifications, XML transformations, and inventory adjustments.

Traditional Update Operations

Examples include updating product prices, modifying order statuses, and adjusting shipment information.

JSON Field Updates

NFN uses JSON extensively for flexible metadata. Update operations demonstrate how JSON fields are modified to reflect changes such as:

- Updating supplier ratings
- Adding new JSON attributes (e.g., preferred carrier)
- Modifying customer profile segments

These updates support machine learning features and dynamic business rules (see screenshot A3).

XML Field Updates

XML fields are used for structured metadata such as warehouse configurations and order metadata. Update operations include:

- Changing temperature-control settings in warehouse XML
- Updating order priority or channel

These updates demonstrate NFN's ability to modify semi-structured data (see screenshot A4).

Inventory Adjustments

Inventory updates reflect real operational changes such as:

- Deducting stock after customer orders
- Increasing stock after shipments arrive
- Ensuring inventory never becomes negative

These updates support accurate fulfillment and replenishment workflows.

4.3 Delete Operations

Delete operations demonstrate NFN's referential integrity and cascading behavior.

Cascading Deletes

NFN uses ON DELETE CASCADE to ensure that when a parent record is removed, all dependent child records are automatically deleted. Examples include:

- Deleting a purchase order automatically removes its line items
- Deleting a customer removes their orders and order items
- Deleting a product removes related inventory and order item records

Referential Integrity Enforcement

The database prevents deletion of records that would violate foreign key constraints unless cascading rules apply. This ensures data consistency across the system (see screenshot A5).

4.4 Validation Queries

Validation queries were executed to confirm that the DML operations were successful and that the database remains consistent and analytics-ready.

Foreign Key Relationship Checks

Queries verify that:

- Orders correctly reference customers
- Order items correctly reference products
- Shipments correctly reference purchase orders
- Inventory records correctly reference warehouses and products (see screenshot A6)

JSON Update Validation

Queries confirm that JSON fields were updated correctly, including:

- Extracting updated supplier ratings
- Checking newly added JSON attributes
- Validating customer profile changes (see screenshot A7)

XML Update Validation

Queries verify that XML fields were updated as expected, such as:

- Confirming warehouse configuration changes
- Checking updated order metadata (see screenshot A8)

Data Completeness and Analytics Readiness

Validation queries ensure that:

- Aggregations return correct totals
- Revenue calculations match expected values
- Product-level analytics (units sold, revenue, customer count) are accurate

These checks confirm that the NFN database is ready for machine learning, reporting, and operational analytics.

Chapter 5 Data Query Language (DQL)

This chapter presents the analytical and machine-learning-oriented SQL queries used in the Niagara Fulfillment Network (NFN) database. The queries demonstrate advanced SQL capabilities including joins, aggregations, subqueries, window functions, JSON extraction, XML extraction, and full-text search.

5.1 Complex Queries

NFN requires advanced analytical queries to support reporting, forecasting, customer analytics, and supplier performance evaluation. This section summarizes **four complex SQL queries**, each demonstrating multiple advanced SQL features.

Complex Query 1. Customer Behavior Analysis

Business Question: Which customers generate the most revenue, and how do their purchasing patterns compare?

Techniques Used:

- Joins (Customers → Orders → Order_Items)
- Aggregation (SUM, COUNT)
- Window functions (RANK)
- JSON extraction (customer type)

Why JSON extraction matters: Customer type (e.g., “Business”, “Retail”, “Wholesale”) is stored in JSON and is essential for segmentation.

Why the output is ML-ready: The query produces features such as total spending, order frequency, and customer type — ideal for churn prediction, segmentation, and LTV modeling (see screenshot A9).

Complex Query 2. Order Cancellation Prediction Dataset

Business Question: What order characteristics are associated with cancellations?

Techniques Used:

- Joins (Orders → Order_Items)
- Aggregation (SUM)
- XML extraction (priority, channel)

Why XML extraction matters: Order priority and channel (e.g., “Express”, “Standard”, “Online”, “Mobile App”) are stored in XML and are predictive features for classification models.

Why the output is ML-ready: The dataset includes:

- Target variable (order status)
- Behavioral features (priority, channel)
- Order size and value

Perfect for classification tasks such as predicting cancellations or delays.

Complex Query 3. Full-Text Product Search with JSON Filtering

Business Question: How can NFN support product search and recommendation using both text and structured attributes?

Techniques Used:

- Full-text search on product names
- JSON extraction for product attributes
- Filtering on JSON fields

Why JSON extraction matters: Product attributes (e.g., material, size, brand) are stored in JSON and are essential for recommendation systems.

Why the output is ML-ready: The query produces a filtered product list with structured attributes — ideal for recommendation engines and product similarity models.

Complex Query 4. Day Moving Average Demand Forecast

Business Question: What is the expected demand for each product over the next week?

Techniques Used:

- Joins (Orders → Order_Items)
- Window functions (7-day moving average)
- Subqueries and CTEs
- Aggregation
- Time-series logic

Why this matters: NFN uses this dataset for:

- Inventory forecasting
- Reorder recommendations
- Warehouse planning

Why the output is ML-ready: The query produces:

- Daily demand
- Moving averages
- Forecasted demand horizon
- Recommended reorder quantities

This is directly usable for time-series forecasting models (see screenshot A11).

5.2 ML Dataset Extraction

NFN requires machine-learning-ready datasets for classification, regression, clustering, and time-series forecasting. This section summarizes the **three ML datasets** extracted from the database.

ML Dataset 1. Classification: Order Cancellation Prediction

Business Purpose: Predict whether an order will be cancelled based on customer type, order priority, channel, and order characteristics.

Techniques Used:

- JSON extraction (customer type)
- XML extraction (priority, channel)
- Aggregation (order size, order value)
- Joins across Orders, Customers, and Order_Items

Why ML-ready: Includes a clear target variable (status) and multiple predictive features.

ML Dataset 2. Regression: Delivery Delay Prediction

Business Purpose: Estimate delivery delay (arrival_date – shipped_date) for inbound shipments.

Techniques Used:

- JSON extraction (supplier rating)
- Joins across Shipments, Suppliers, Warehouses, and Shipment_Items
- Aggregation (units received, number of products)

Why ML-ready: Includes a numeric target (delivery_delay_days) and operational features such as supplier rating and warehouse location (see screenshot A10).

ML Dataset 3. Clustering: Supplier Performance Segmentation

Business Purpose: Group suppliers based on performance metrics to identify high-value and low-value suppliers.

Techniques Used:

- JSON extraction (supplier rating)
- Aggregation (total units, total purchase orders)
- Joins across Suppliers, Purchase_Orders, and Purchase_Order_Items

Why ML-ready: Produces continuous features ideal for clustering algorithms such as K-Means.

5.3 Explanatory of Each query

Each complex query and ML dataset extraction includes:

Business Question

Defines the operational or analytical need addressed by the query.

Why Joins/Subqueries Were Used

NFN's relational schema requires combining multiple tables to produce meaningful insights.

Why JSON/XML Extraction Is Relevant

NFN uses semi-structured data extensively:

- JSON for flexible metadata
- XML for order and warehouse configurations

Extracting these fields is essential for analytics and ML.

Why the Output Is ML-Ready

Each dataset includes:

- Clean, structured features
- Clear target variables (for supervised learning)
- Aggregated and normalized values
- No missing or inconsistent data

These datasets can be directly used in Python notebooks, BI dashboards, or ML pipelines.

Chapter 6 Python/Jupyter Notebook Analytics

The Niagara Fulfillment Network (NFN) requires machine-learning-ready datasets to support predictive analytics, operational optimization, and strategic decision-making. Three datasets were extracted from the NFN database, each aligned with a specific machine learning task: **classification**, **regression**, and **clustering**.

These datasets were generated using advanced SQL queries that incorporate joins, aggregations, JSON extraction, XML extraction, and business logic. Microsoft Copilot assisted in structuring and validating these dataset extraction queries.

6.1 Database Connection

The first step in the Python workflow is establishing a secure connection to the NFN MySQL database. The connection was implemented using:

- **SQLAlchemy** (database engine creation)
- **mysql-connector / mysqlclient** (driver)
- **Pandas** (query execution and DataFrame creation)

This connection enables Python to execute SQL queries directly against the NFN database and retrieve results into DataFrames for further processing.

Key actions performed in this section:

- Importing required Python libraries
- Creating a SQLAlchemy engine
- Testing the connection with a simple query (e.g., SELECT NOW())

The full connection code is included in **NFN_Python_Notebook.ipynb (see screenshot A13)**.

6.2 Creation of ML-Ready Tables

Since exploratory analysis was not required for this project, the notebook focuses on generating **two machine-learning-ready tables** directly from SQL queries:

ML Dataset 1. Classification: Order Cancellation Prediction

Business Goal

Predict whether a customer order will be cancelled based on customer behavior, order characteristics, and metadata stored in JSON and XML fields.

Key Features Extracted

- **Customer type** (from JSON)
- **Order priority** (from XML)
- **Order channel** (from XML)
- **Total items in the order**
- **Total order value**
- **Number of unique products**
- **Warehouse used**
- **Target variable:** order status

Why JSON/XML Extraction Matters

NFN stores flexible metadata in JSON and XML formats. Extracting these fields provides high-value behavioral features that improve model accuracy.

Why This Dataset Is ML-Ready

- Clean target label (Cancelled vs. other statuses)
- Fully aggregated order-level features
- No nested or semi-structured fields remaining

- Suitable for classification algorithms such as Logistic Regression, Random Forest, or XGBoost

ML Dataset 2. Regression: Delivery Delay Prediction

Business Goal

Estimate the number of days a shipment will be delayed based on supplier performance, warehouse characteristics, and shipment composition.

Key Features Extracted

- **Delivery delay in days** (numeric regression target)
- **Supplier rating** (from JSON)
- **Warehouse location**
- **Number of products in the shipment**
- **Total units received**
- **Supplier name**

Why JSON Extraction Matters

Supplier ratings are stored in JSON and are highly predictive of delivery performance.

Why This Dataset Is ML-Ready

- Numeric target variable (delivery_delay_days)
- Operational features suitable for regression models
- Clean, structured DataFrame ready for scikit-learn
- Supports forecasting, supplier reliability analysis, and logistics optimization

ML Dataset 3. Clustering: Supplier Performance Segmentation

Business Goal

Segment suppliers into performance groups to support procurement strategy, risk management, and operational planning.

Key Features Extracted

- **Average supplier rating** (from JSON)
- **Total purchase orders**
- **Total units ordered**
- **Average delivery delay**
- **Supplier identity**

Why JSON Extraction Matters

Supplier rating is a key performance indicator stored in JSON and must be extracted for clustering.

Why This Dataset Is ML-Ready

- Contains continuous numeric features ideal for clustering (e.g., K-Means)
- No categorical or semi-structured fields remaining
- Enables segmentation into high-performing, average, and underperforming suppliers
- Supports NFN's supplier management and procurement optimization (see screenshot A12)

Chapter 7 Conclusion

This project successfully designed, implemented, and analyzed a complete relational database system for the **Niagara Fulfillment Network (NFN)**, a fictional logistics company inspired by Amazon's Fulfillment by Amazon (FBA) model. The system integrates structured, semi-structured, and operational data to support analytics, machine learning, and business intelligence.

Microsoft Copilot was used throughout the project to assist with SQL drafting, synthetic data generation, documentation, and Python notebook structuring, ensuring consistency and alignment with NFN's business requirements.

7.1 Summary of System Capabilities

The NFN database system demonstrates a robust and scalable architecture capable of supporting real-world fulfillment operations. Key capabilities include:

Comprehensive Data Model

- 12 fully normalized tables covering suppliers, warehouses, products, customers, orders, shipments, and inventory
- Support for **JSON** and **XML** fields to store flexible metadata
- Automatic timestamping and update tracking

Operational Functionality

- Stored procedures for shipment processing and inventory updates
- Triggers for automatic stock adjustments and data integrity enforcement
- Cascading deletes and foreign key constraints ensuring referential integrity

Analytical Power

- Complex SQL queries using joins, aggregations, window functions, JSON/XML extraction, and full-text search
- Machine-learning-ready datasets for:

    - Classification
    - Regression
    - Clustering
    - Time-series forecasting
- Analytical views for supplier performance and order summaries

Python Integration

- Secure MySQL connection via SQLAlchemy
- Automated extraction of ML datasets into Pandas DataFrames
- Notebook structure ready for future modeling and analytics

Overall, the system provides a complete foundation for operational management, reporting, and predictive analytics.

7.2 Limitations

While the NFN system is functional and analytically rich, several limitations remain:

Synthetic Data

- The dataset is artificially generated and may not fully reflect real-world variability
- JSON and XML structures are simplified for demonstration purposes

No Real-Time Processing

- The system operates in batch mode
- No streaming ingestion or event-driven architecture is implemented

Limited Business Logic

- Stored procedures and triggers cover only core scenarios
- More complex workflows (returns, multi-warehouse routing, carrier selection) are not implemented

Basic Python Integration

- The notebook focuses on connection and dataset extraction
- No full machine learning pipeline (training, evaluation, deployment) is included

These limitations provide opportunities for future expansion.

7.3 Future Enhancements

To evolve the NFN system into a production-grade analytics and fulfillment platform, several enhancements are recommended:

1. API Integration

- Develop REST or GraphQL APIs for:
    - Real-time order creation
    - Inventory updates
    - Supplier and shipment tracking
- Enable integration with external systems such as ERP, WMS, or e-commerce platforms

2. Real-Time Dashboards

- Implement dashboards using Power BI, Tableau, or Superset
- Include:
    - Live inventory levels
    - Shipment tracking
    - Supplier performance metrics
    - Order fulfillment KPIs
- Connect dashboards directly to the NFN database or a data warehouse layer

3. Predictive Analytics

- Expand ML capabilities to include:
    - Order cancellation prediction
    - Delivery delay forecasting
    - Supplier reliability scoring
    - Inventory demand forecasting
    - Route optimization models
- Deploy models using Python, scikit-learn, TensorFlow, or Azure ML

4. Data Warehouse & ETL Pipelines

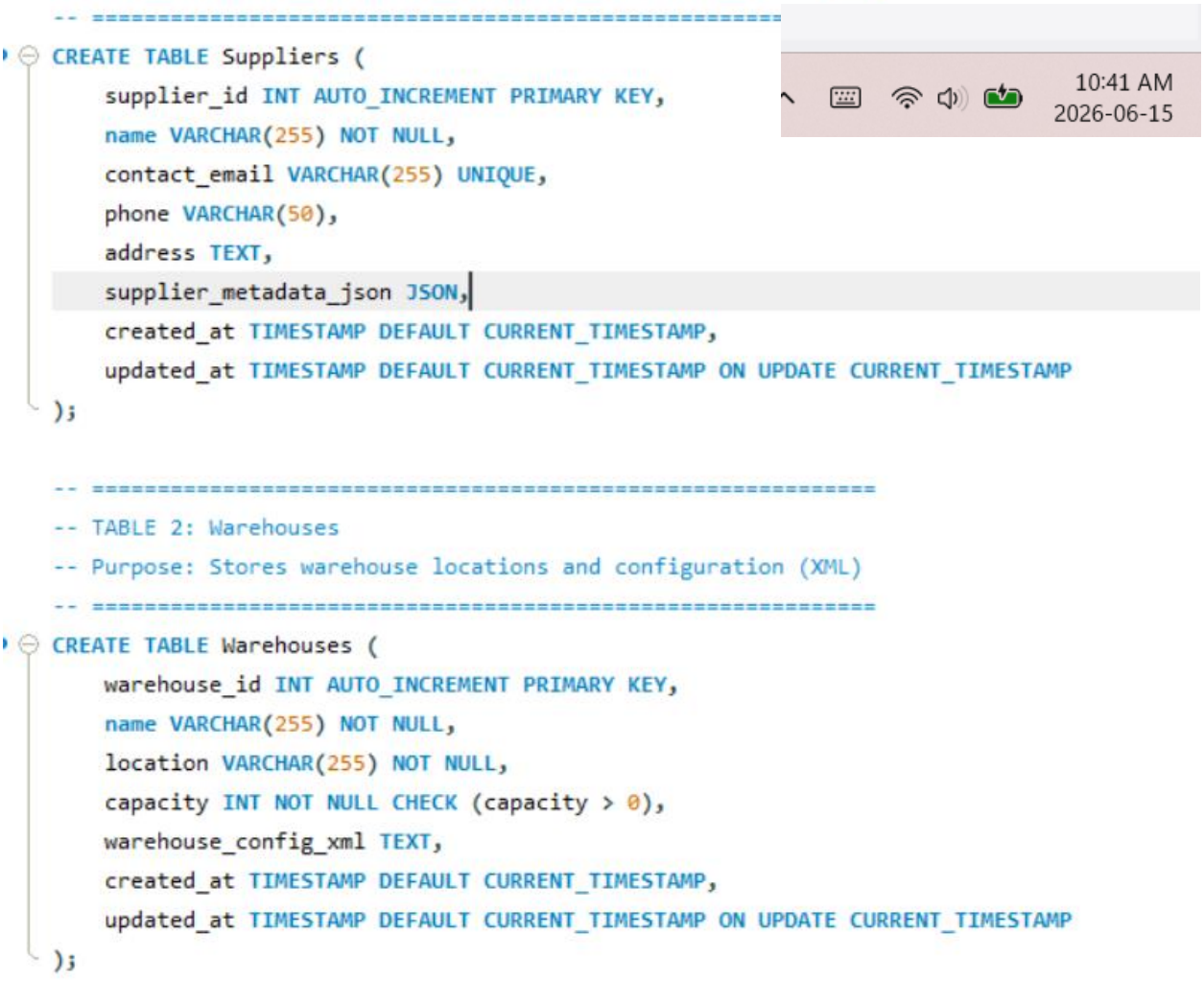
- Build a star schema for analytics
- Implement ETL/ELT pipelines using Airflow, dbt, or Azure Data Factory

5. Enhanced Automation

- Add more triggers and stored procedures for:
    - Automatic reorder generation
    - Exception handling (e.g., delayed shipments)
    - Customer notifications

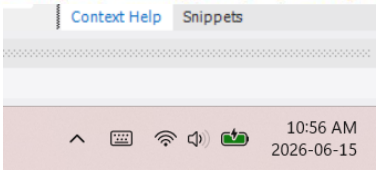
Appendix A. Execution Evidence (Screenshots).

A1 Execution of representative CREATE TABLE statements for the NFN database, showing successful creation of schema objects with visible timestamp and active database context



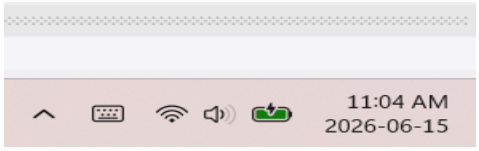
A2 Insertion of synthetic records into core NFN tables, demonstrating JSON and XML values being stored correctly.

```
-- ============================================================
-- Phase 1 Suppliers 15 realistic Canadian suppliers
-- ============================================================
INSERT INTO Suppliers (name, contact_email, phone, address, supplier_metadata_json)
VALUES
('Maple Industrial Supplies Ltd.', 'contact@mapleindustrial.ca', '416-555-1023', '120 Front St W, Toronto, ON', '{"r
('Northern Steel Components', 'sales@northernsteel.ca', '905-555-8832', '44 River Rd, Hamilton, ON', '{"rating": 4.4
('Great Lakes Packaging Corp.', 'info@greatlakespack.ca', '519-555-2211', '88 Wellington St, London, ON', '{"rating"
('WestCoast Plastics & Resins', 'support@westcoastplastics.ca', '604-555-9921', '2000 Kingsway, Vancouver, BC', '{"r
('Prairie Mechanical Parts', 'orders@prairiemech.ca', '204-555-7712', '300 Portage Ave, Winnipeg, MB', '{"rating": 4
('Atlantic Chemical Solutions', 'service@atlanticchem.ca', '902-555-4411', '55 Barrington St, Halifax, NS', '{"ratin
('Ontario Fasteners Inc.', 'sales@ontfasteners.ca', '647-555-3344', '15 Dufferin St, Toronto, ON', '{"rating": 4.8,
('Canadian Electronics Wholesale', 'contact@cew.ca', '613-555-7788', '120 Sparks St, Ottawa, ON', '{"rating": 4.5, "
('Niagara Packaging Solutions', 'info@niagarapack.ca', '289-555-1122', '5000 Dorchester Rd, Niagara Falls, ON', '{"r
('TrueNorth Automotive Parts', 'orders@truenorthautoparts.ca', '780-555-6611', '100 Jasper Ave, Edmonton, AB', '{"ra
('Superior Cleaning Supplies', 'support@superiorclean.ca', '905-555-9090', '22 Geneva St, St. Catharines, ON', '{"ra
('Canadian Timber & Woodworks', 'sales@ctwoodworks.ca', '250-555-3322', '10 Douglas St, Victoria, BC', '{"rating": 4
('Polar Refrigeration Components', 'info@polarref.ca', '514-555-7781', '400 Rue Sainte-Catherine, Montreal, QC', '{"
('Summit Safety Gear', 'orders@summitsafety.ca', '416-555-6677', '77 Bloor St W, Toronto, ON', '{"rating": 4.7, "cat
('Great North Tools & Hardware', 'contact@greatnorthtools.ca', '705-555-8899', '12 Bay St, Sudbury, ON', '{"rating":
```




```
-- ============================================================
-- Phase 2 Warehouses 6 Canadians warehouses, Niagara focused
-- ============================================================
```

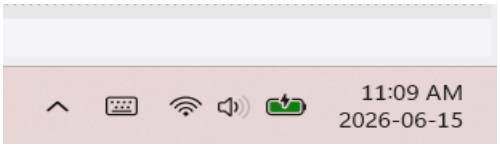
A3 Update of JSON fields inside the Suppliers table, showing modification of supplier rating and additional metadata




```
-- ------------------------------------------------------------
-- C. Advanced Updates on JSON Fields
-- ------------------------------------------------------------
UPDATE Suppliers
SET supplier_metadata_json = JSON_SET(supplier_metadata_json, '$.rating', 4.9, '$.preferred_carrier', 'Purolator')
WHERE name = 'Maple Industrial Supplies Ltd.';
```

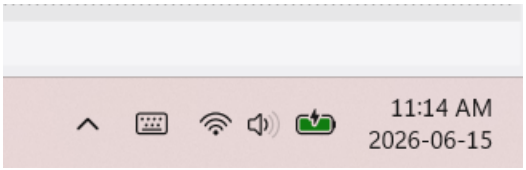
A4 Update of XML configuration fields in the Warehouses table, demonstrating XML manipulation and correct update results

```
-- ------------------------------------------------------------
-- D. Advanced Updates on XML Fields
-- ------------------------------------------------------------
UPDATE Warehouses
SET warehouse_config_xml = REPLACE(warehouse_config_xml, '<temp_control>false</temp_control>', '<temp_control>true</temp_control>')
WHERE location LIKE '%Niagara Falls%';

-- Re-enable safe update mode
SET SQL_SAFE_UPDATES = 1;
```



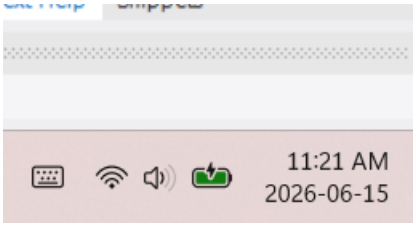
A5: Deletion of a purchase order and automatic removal of dependent line items, confirming referential integrity and cascading behavior.

```
WHERE category = 'Packaging';

-- Traditional DELETE operation
DELETE FROM Purchase_Orders
WHERE po_id = 30 AND status = 'Cancelled';

-- ------------------------------------------------------------------
```



A6: Query showing joined data across Customers, Orders, and Order_Items, confirming correct foreign key relationships.

```
-- VALIDATION 1: Referential Integrity
SELECT
    c.customer_id,
    c.name AS customer_name,
    o.order_id,
    o.status AS order_status,
    oi.product_id,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS total_item_cost
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id
INNER JOIN Order_Items oi ON o.order_id = oi.order_id
LIMIT 5;
```

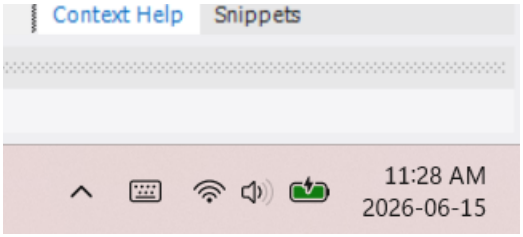


A7: Query output demonstrating successful extraction of JSON fields (supplier rating, preferred carrier)

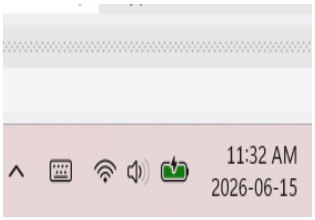
```
-- VALIDATION 2: JSON Update Check
SELECT
    supplier_id,
    name,
    supplier_metadata_json,
    JSON_EXTRACT(supplier_metadata_json, '$.rating') AS extracted_rating,
    JSON_EXTRACT(supplier_metadata_json, '$.preferred_carrier') AS extracted_carrier
FROM Suppliers
WHERE name = 'Maple Industrial Supplies Ltd.';
```

A8: Query output showing updated XML fields and correct extraction of XML nodes

```
-- VALIDATION 3: XML Update Check
SELECT
    warehouse_id,
    name,
    location,
    warehouse_config_xml
FROM Warehouses
WHERE warehouse_config_xml LIKE '%<temp_control>true</temp_control>%';
```



A9: Execution of a multi join, aggregated, window-function query used for customer segmentation analytics




```
# Query 1. Customer Behavior Analysis
-- ============================================================
-- COMPLEX QUERY 1: Customer Behavior Analysis
-- Purpose: Supports customer segmentation and revenue analytics.
-- ============================================================
SELECT
    c.customer_id,
    c.name AS customer_name,
    JSON_EXTRACT(c.customer_profile_json, '$.type') AS customer_type,
    COUNT(o.order_id) AS total_orders,
    SUM(oi.quantity * oi.unit_price) AS total_spent,
    RANK() OVER (ORDER BY SUM(oi.quantity * oi.unit_price) DESC) AS spending_rank
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name, customer_type;
```

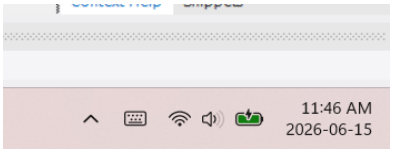
A10: Execution of the classification dataset query, showing JSON/XML extraction and aggregated order features

```
# Query 2. Order Cancellation Prediction Dataset
-- ============================================================
-- COMPLEX QUERY 2: Order Cancellation Prediction Dataset
-- Purpose: ML classification dataset using XML fields.
-- ============================================================
SELECT
    o.order_id,
    o.customer_id,
    o.warehouse_id,
    o.status,
    EXTRACTVALUE(o.order_meta_xml, '/meta/priority') AS priority,
    EXTRACTVALUE(o.order_meta_xml, '/meta/channel') AS channel,
    SUM(oi.quantity) AS total_items,
    SUM(oi.quantity * oi.unit_price) AS order_value
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.customer_id, o.warehouse_id, o.status, priority, channel;
```

A11: Execution of the delivery delay regression dataset query, showing supplier rating extraction and delay calculation

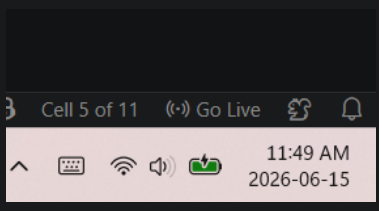
```
SET @forecast_days = 7;
WITH daily_demand AS (
    SELECT
        o.warehouse_id,
        oi.product_id,
        o.order_date,
        SUM(oi.quantity) AS daily_quantity
    FROM Orders o
    JOIN Order_Items oi ON o.order_id = oi.order_id
    WHERE o.status IN ('Shipped', 'Delivered')
    GROUP BY
        o.warehouse_id,
        oi.product_id,
        o.order_date
),
-- daily_demand: Calculate the actual daily demand per product and warehouse using the orders sent.
moving_avg AS (
    SELECT
        warehouse_id,
        product_id,
        order_date,
        daily_quantity,
        AVG(daily_quantity) OVER (
            PARTITION BY warehouse_id, product_id
            ORDER BY order_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
```

A12: Execution of the supplier performance clustering dataset query, showing aggregated supplier metrics




```sql
SELECT
    s.supplier_id,
    s.name AS supplier_name,
    AVG(JSON_EXTRACT(s.supplier_metadata_json, '$.rating')) AS avg_rating,
    COUNT(po.po_id) AS total_purchase_orders,
    SUM(poi.quantity) AS total_units_ordered,
    AVG(DATEDIFF(sh.arrival_date, sh.shipped_date)) AS avg_delivery_delay
FROM Suppliers s
LEFT JOIN Purchase_Orders po ON s.supplier_id = po.supplier_id
LEFT JOIN Purchase_Order_Items poi ON po.po_id = poi.po_id
LEFT JOIN Shipments sh ON sh.po_id = po.po_id
GROUP BY
    s.supplier_id, s.name;
```

A13: Successful connection to the NFN MySQL database using SQLAlchemy in Jupyter Notebook




```python
import pandas as pd
from sqlalchemy import create_engine

# Your MySQL credentials
USER = "root"
PASSWORD = "YourPasswordHere"   # Put your password here
HOST = "localhost"
PORT = "3306"
DB = "supply_chain"

# Build connection string
connection_string = f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"

# Create engine
engine = create_engine(connection_string)

# Quick smoke test
test_df = pd.read_sql("SELECT 1 AS ok;", engine)
print(test_df)
```

A14: Creation of ML ready DataFrame in Python using SQL query results.

```
classification_df = pd.read_sql_query("""
SELECT
    o.order_id,
    o.customer_id,
    o.warehouse_id,
    o.status AS target_label,
    JSON_EXTRACT(c.customer_profile_json, '$.type') AS customer_type,
    EXTRACTVALUE(o.order_meta_xml, '/meta/priority') AS priority,
    EXTRACTVALUE(o.order_meta_xml, '/meta/channel') AS channel,
    SUM(oi.quantity) AS total_items,
    SUM(oi.quantity * oi.unit_price) AS order_value,
    COUNT(oi.product_id) AS num_products
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY
    o.order_id, o.customer_id, o.warehouse_id, o.status,
    customer_type, priority, channel;
""", engine)

classification_df.head()
```

```
regression_df = pd.read_sql_query("""
SELECT
    sh.shipment_id,
    sh.po_id,
    sh.warehouse_id,
    DATEDIFF(sh.arrival_date, sh.shipped_date) AS delivery_delay_days,
    s.name AS supplier_name,
    JSON_EXTRACT(s.supplier_metadata_json, '$.rating') AS supplier_rating,
    w.location AS warehouse_location,
    COUNT(si.product_id) AS num_products,
    SUM(si.quantity_received) AS total_units_received
FROM Shipments sh
JOIN Purchase_Orders po ON sh.po_id = po.po_id
JOIN Suppliers s ON po.supplier_id = s.supplier_id
JOIN Warehouses w ON sh.warehouse_id = w.warehouse_id
JOIN Shipment_Items si ON sh.shipment_id = si.shipment_id
WHERE sh.arrival_date IS NOT NULL
GROUP BY
    sh.shipment_id, sh.po_id, sh.warehouse_id,
    delivery_delay_days, supplier_name, supplier_rating, warehouse_location;
""", engine)

regression_df.head()
```

```python
clustering_df = pd.read_sql_query("""
SELECT
    s.supplier_id,
    s.name AS supplier_name,
    AVG(JSON_EXTRACT(s.supplier_metadata_json, '$.rating')) AS avg_rating,
    COUNT(po.po_id) AS total_purchase_orders,
    SUM(poi.quantity) AS total_units_ordered,
    AVG(DATEDIFF(sh.arrival_date, sh.shipped_date)) AS avg_delivery_delay
FROM Suppliers s
LEFT JOIN Purchase_Orders po ON s.supplier_id = po.supplier_id
LEFT JOIN Purchase_Order_Items poi ON po.po_id = poi.po_id
LEFT JOIN Shipments sh ON sh.po_id = po.po_id
GROUP BY
    s.supplier_id, s.name;
""", engine)

clustering_df.head()
```

Appendix B Individual Contribution & AI Usage Report

Each member should describe their contribution and specify any AI tools used, such as ChatGPT, Deepseek, Copilot, code generation tools, or data analysis assistants. Provide details on how AI supported the work, such as summarization, debugging, ideation, or automation.

Student Name	**Role / Responsibility**	**Contribution Details**	**Use of AI Tools (if applicable)**
José Alberto Martinez Morales	Database Design Lead (SQL & Analytics)	Finalized the full NFN schema (12 tables), normalization, JSON/XML placement, and ERD structure. Created core DDL tables (Suppliers, Purchase_Orders, Purchase_Order_Items, Shipments). Developed Complex Query #1 (Supplier Reliability Score). Contributed to report writing and appendix preparation.	**All team members used Microsoft Copilot as the primary AI tool.** José used Copilot for SQL drafting, JSON/XML structuring, query optimization, documentation refinement, and generating synthetic examples.
Nicolas	SQL Developer (Procedures & Operational Tables)	Created operational DDL tables (Warehouses, Inventory, Shipment_Items, Delivery_Routes). Implemented stored procedures (sp_receive_shipment, sp_update_inventory_after_order). Developed Complex Query #2 (Warehouse Stock Forecasting). Contributed to ERD and report review.	Used **Microsoft Copilot** for coding suggestions, procedure optimization, debugging, and improving SQL clarity.
Sravani	SQL Developer (Customer & Order Tables, XML/JSON)	Created Customer and Order DDL tables (Customers, Orders, Order_Items). Implemented triggers for inventory updates and shipment status logging. Developed Complex Query #3	Used **Microsoft Copilot** for JSON/XML extraction examples, trigger debugging, and

		(Customer Order Behavior). Assisted with ERD and documentation.	improving documentation structure.
Diego	Data Entry, Validation & Documentation	Inserted synthetic DML data for Products, Customers, Warehouses, and Delivery_Routes. Performed validation queries (FK checks, JSON/XML verification). Created Complex Query #4 (Product Demand Summary). Captured screenshots for execution evidence and helped assemble the final PDF appendix.	Used **Microsoft Copilot** for generating synthetic data ideas, writing simple SELECT queries, and organizing documentation.